DAGSTER · DELTA-RS · DUCKDB · KAFKA · AVRO

Jobotron: the RFC's upstream producer

Candidate selection on the lakehouse — delta-rs scans, DuckDB joins,
one Avro message per customer, routed to per-operator Kafka topics.

TL;DR Execution Summary

Runs candidate-selection SQL directly on the lake — column-pruned delta-rs scans registered in DuckDB. Zero warehouse compute: **0.0 DBUs**, asserted by the cost meter.

Publishes **one Avro message per selected customer** to that customer's operator topic — the schema is derived from the query and registered in Apicurio.

Stateless by design: every run republishes the full eligible set. **Dedup and re-validation belong to the worker.**

The use case, per the RFC

> Scheduling and candidate selection remain externalized to upstream producers. Such producers can be scheduled Databricks jobs that query Data Lake to identify eligible customers and publish individual messages to the relevant operator topic.
>
> — Jobotron RFC, 08.10.2026 (Accounts & Payments / KYC solutions)

The RFC's Jobotron

Worker service with per-operator Kafka consumers, a Job Status WebAPI and a state database — Redis idempotency, eligibility re-validation, Iris / CustomerService actions. **Owned by Accounts & Payments.**

This component — the upstream producer

Scheduled candidate selection + publishing on Dagster. The RFC's review thread asked for exactly this: *"Could the RFC name Dagster as the scheduler and publisher?"* **Owned by Big Data.**

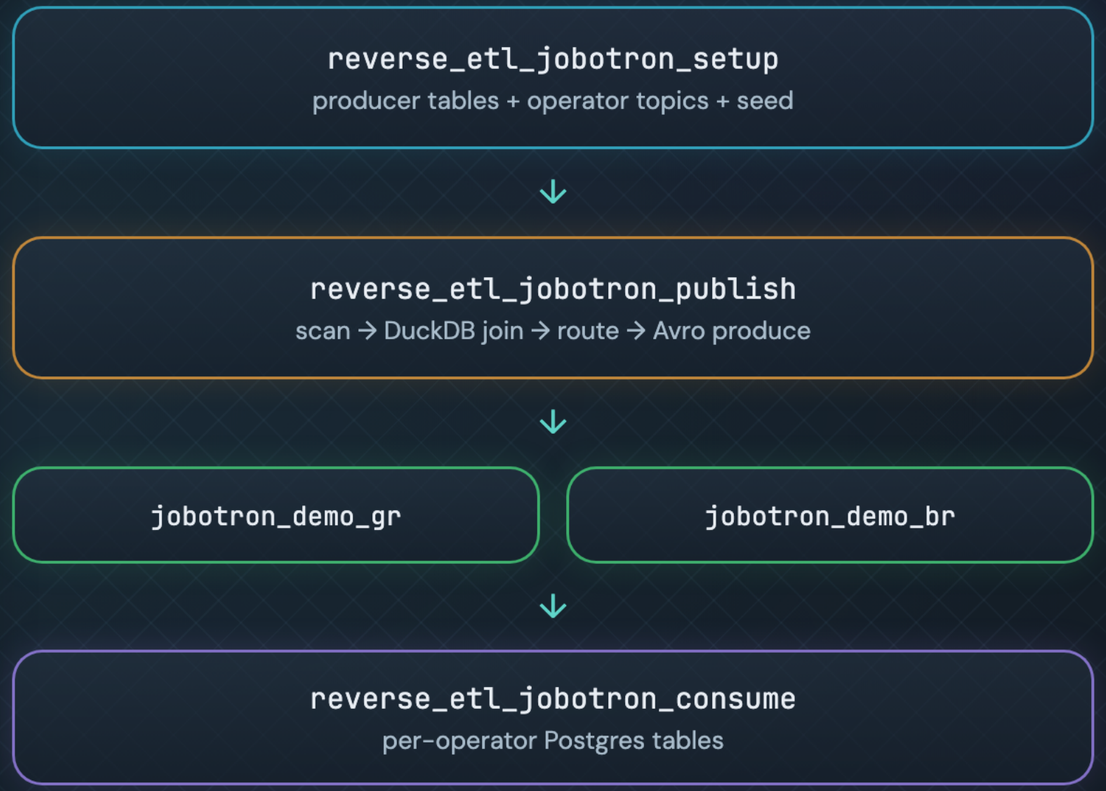
The unit of work: one Kafka message per selected customer. The worker verifies it has not been processed before, re-validates eligibility against live data, executes the actions and records the outcome.

End-to-end: Delta lakehouse to per-operator topics

The YAML contract, and the assets it generates

jobotron.yaml

```yaml
# orchestration/defs/reverse_etl_jobotron/defs.yaml
name:
 kyc_unverified_pm
job_type:
 kyc.unverified_pm
schedule:
 "0 * * * *"
sources:
  customers:
  de_dev.sr_poc_external.customers
    columns: [customer_id, email, phone_number,
              locale, is_active]
  kyc_cases:
    de_dev.sr_poc_external.kyc_cases
    columns: [customer_id, status, operator]
query:
 |
  SELECT c.customer_id, c.email, c.locale, k.operator
  FROM customers AS c JOIN kyc_cases AS k
  WHERE k.status = 'PENDING' AND c.is_active
key_columns:
 [customer_id]
operator_column:
 operator
topic_template:
 "jobotron-{operator}"
```



One YAML → the whole asset graph. The **ReverseEtlJobotron** component generates setup / publish / consume; the health check and the

What one publish run does — seven stages

1

Resolve

table locations via the cached UC REST helper

2

Scan

column-pruned delta-rs snapshots → pyarrow

3

Query

DuckDB over registered snapshots — joins included

4

Route & stamp

topic from operator_column; run_id + job_type stamped

5

Schema

Avro derived from the result shape; ensured in Apicurio

6

Produce

one message per customer, key = customer_id; lz4 + backpressure

7

Record

rows, per-operator counts, duration — warehouse_seconds: 0.0

Every stage is **warehouse-free**. The only Databricks touch is the control plane.

The engine decision

OPTION	SQL POWER	DELTA ACCESS	JVM	VERDICT
delta-rs scan → DuckDB over the snapshots	full SQL incl. joins	delta-rs — proven on our tables	no	RECOMMENDED
DuckDB + its `delta` extension	full SQL + pushdown	bundles its own kernel reader	no	**later optimization**
Daft / polars	partial SQL	delta-rs based	no	**dialect limits on the joins**
Databricks job (the RFC's wording)	full SQL	native	—	**meters DBUs — the RFC's own con**

Multi-table joins are in scope from day one — the RFC's whole point is moving complex selection queries off VOperDB. Day-one trade-off: no predicate pushdown into the scans, mitigated by column pruning and a sizing spike.

Databricks job path

the RFC's literal wording

≈ $10 / month / task

`warehouse_seconds > 0`

Producer on Dagster

this component

0.0 DBUs — measured

`warehouse_seconds: 0.0 asserted`

E2E on a laptop: the full path — YAML → lake → Kafka → Apicurio — runs against the local demo stack. The RFC review thread's testing concern, answered.

The Phase-1 demo — built and verified live

~5 s end to end

scan → join → route → produce

0.0 warehouse seconds

asserted by the cost meter

4 messages / run

gr: 1001, 1002 · br: 1004, 1005

2 plain-Delta tables → `publish` → `jobotron_demo_gr · br` → `consume` → per-operator tables

The money moment, twice

Add an unverified customer → publish → it appears in its operator's table only. **Verify a customer** → publish → it disappears. No tombstones — full-set semantics in action.

The orchestrated lifecycle

Setup

Idempotent `IF NOT EXISTS` creation of the two producer tables, the operator topics and the seed segment.

Publish

Scan → join → route → produce. Manual from the demo page, or scheduled by the YAML's cron.

Consume

The demo consumer drains each operator topic and rebuilds the per-operator Postgres tables from the latest run only.

Check

Producer health check + cost meter: `warehouse_seconds 0.0` asserted on every run.

Every step is idempotent — re-run on any failure. Reset chain: reset → setup → first publish → first consume.

Critical trade-offs & implementation notes

Full set, no tombstones

Every run republishes whoever matches **right now**. A customer leaving the segment is silence, not a delete — the worker's dedup and re-validation make that correct by construction.

GDPR by construction

The Avro payload is exactly the YAML's SELECT. TLS on the prod Kafka path, 10-day topic retention — and email / phone stay out of any job that does not need them.

Operator isolation

Routing never mixes markets: `topic_template` + `operator_column`, and an unknown operator fails the row loudly instead of publishing cross-market.

OPEN QUESTIONS (RFC §8)

Schema-contract ownership (Big Data publishes, A&P approves) · scan-sizing spike on multi-million-row tables · prod Kafka TLS · customer-id namespacing across operators · full-set vs delta-of-set.

VERIFICATION: 0 DBU · NO JVM · CONTRACT-CORRECT MESSAGES · E2E ON A LAPTOP · HOUR-SAFE